BiasClean Justice Domain

Phase 1: Consolidation & Regression Testing

v3.0.0 → v3.6.1

AI Fairness CIO
Prepared by: Hamid Tavakoli
July 30, 2026

1. Executive Summary

This report documents Phase 1 of the BiasClean™ Justice-domain hardening effort: the consolidation and regression-testing work that took the pipeline from its first production release (v3.0.0) through v3.6.1, immediately before Phase 2's external validation against six real datasets began. Where Phase 2 (see "BiasClean External Validation in Justice Domain") tests the pipeline against real-world data, Phase 1 is the record of how the pipeline itself was built into something worth testing: 15 releases, each driven by a specific, root-caused finding, almost all surfaced on real data (principally COMPAS, with NIJ and North Carolina surfacing three of the later fixes) rather than synthetic test cases alone.

Three findings recur across this history and are worth naming up front, since they explain why later releases exist:

- Silent inconsistency between what audit_trail.json's machine-readable numbers say and what report.pdf's plain-language summary says, in either direction, was the single most common bug class, found and fixed five separate times across this range (v3.1.2, v3.1.3, v3.2.2, v3.5.1, v3.6.1).
- Every fairness-mitigation mechanism in the pipeline rebalancing, SVM enforcement, and the fairness regulator was tested and hardened against real COMPAS data, not assumed correct from synthetic cases alone; several serious defects (SVM making bias worse while reporting an improvement; a reversal crossing past parity) were caught only because real data was used.
- The pipeline's mitigation safeguards follow a deliberate two-layer design, introduced in v3.4.0 and referred to throughout this document by the terms coined for it at the time: a proactive "thermostat" (the regulator, which caps a correction before it can overshoot) and a reactive "fuse" (the gate, which flags and reverts when the thermostat's local view couldn't see a problem).

2. Methodology

Every entry in this report follows the same structure: what was found (and how), the root cause traced to an exact code path, the fix, and how it was verified; the same protocol Phase 2 continues. Two conventions recur:

- "Found by" names the specific real dataset or regression check that surfaced the issue wherever one exists, rather than describing findings in the abstract. Several early entries (v3.1.1, v3.3.1, v3.4.0) were found through direct instrumentation of what the classifier was actually training on, rather than through assumption.
- Where a release changed default behavior rather than fixing a defect (e.g. switching the default rebalancing method), this is stated explicitly as a methodology change, not a bugfix, matching how the pipeline's own changelog distinguishes the two.

3. Reporting Consolidation & Infrastructure

v3.0.0 → v3.1.0

The foundational consolidation release. Reporting was reduced from 14 scattered output files down to three: report.pdf (plain-language summary plus an always-visible technical detail section), corrected_dataset.csv, and one merged audit_trail.json, the file structure every dataset in Phase 2 was validated against.

> **Fix:** The PDF engine was switched from WeasyPrint to xhtml2pdf. WeasyPrint requires native OS-level libraries (Pango/Cairo/GDK-Pixbuf) that pip cannot provide, which had caused repeated real-world failures including total report loss; xhtml2pdf is pure Python, so a successful pip install guarantees rendering works.
>
> **Fix:** corrected_dataset.csv no longer carries internal bookkeeping columns (exact duplicates of the target column) that had been silently exported alongside the real data.
>
> **Note:** This release also added _detect_mapping_conflicts, surfacing when multiple columns tie for the same protected feature or outcome variable, the same "mapping ties detected" mechanism every Phase 2 report

references, and the first acknowledgment in this codebase's own history that tie resolution (by column position, not confidence) was silent and worth surfacing.

4. Target & Outcome-Column Detection Hardening

v3.1.0 → v3.1.1

Found by: The NIJ Recidivism Forecasting Challenge dataset, the same dataset Phase 2 later re-validated as dataset #3 of 6.

Root cause: Auto-detection could silently pick a meaningless bookkeeping column (a train/test split flag such as 'Training_Sample') as the target instead of the real outcome column. Whole-token boundary matching, added to stop short patterns like “id” or “age” from matching inside unrelated words, also broke deliberate word-stem patterns like “recid” and “violat” from matching their own full-word forms, so a column literally named Recidivism_Within_3years failed to match the 'recid' pattern.

Fix: _pattern_matches gained a prefix_ok flag, used only for outcome-tier detection, anchoring the left boundary but not the right. A defense-in-depth bookkeeping-column denylist (train/test/split/fold/holdout/partition) was also added to the binary-column fallback search, so a similar mistake can't happen even for a future dataset whose real outcome column matches no pattern at all.

Note: Without this fix, the original run on this dataset produced a false “no bias found” result; the pipeline was actually auditing bias in train/test split assignment, not recidivism.

5. SVM-Stage Score Consistency, A Recurring Bug Class

Four separate releases across this range fixed variations of the same underlying problem: a number shown to the reader (report.pdf, or a top-level audit_trail.json field) disagreeing with the number the pipeline actually used internally, specifically around the SVM stage. Grouped here because the pattern itself, not any single instance, is the finding worth understanding.

v3.1.1 → v3.1.2

Found by: Real COMPAS data, SVM enforcement enabled.

Root cause: diagnostic_results['final_bias_score'] was only ever set once, immediately after rebalancing. The SVM block updated its own stage tracker but never touched diagnostic_results, so two different numbers both labeled “final” existed side by side in the same audit_trail.json, and report.pdf read the stale, better-looking one.

Fix: SVM enforcement now updates diagnostic_results directly. An explicit svm_enforcement block (pre_svm_bias_score, post_svm_bias_score, worsened_fairness) was added so this is visible even to someone reading only the machine-readable output.

Note: On the real run, this fix corrected; SVM's own validation accuracy was 54.2%, barely better than a coin flip , and the true final bias score was 0.464, worse than the original unmitigated data (0.095), while the report had confidently displayed “+31% improvement.” This release also flagged, without yet fixing, that SVM enforcement at the time had no actual fairness constraint in training, the gap that v3.3.0's Smart SVM (Section 9) later closed.

v3.1.2 → v3.1.3

Found by: The real COMPAS SVM re-run validating the v3.1.2 fix above.

Root cause: report.pdf's “Why this recommendation” section is built solely from the pre-mitigation audit's rationale, computed before rebalancing or SVM ever ran, so the v3.1.2 fix's own SVM-worsened-fairness finding was never fed back into it.

> **Fix:** generate_report now appends an explicit plain-language reason whenever SVM enforcement worsened fairness, in both legacy and audit-first mode.

v3.2.1 → v3.2.2

> **Found by:** Real COMPAS data.
>
> **Root cause:** bias_scores.final was computed on a different, non-comparable scale than bias_scores.initial whenever the SVM ran. The engine's normal scoring is a raw weighted sum; the SVM stage's tracker call passed None for the score, triggering a different fallback formula, a weighted average divided by 0.45 (Justice's summed weights for 3 present features), inflating the number by roughly 2.2x. Confirmed bit-for-bit: the correct formula gave 0.132 on the exact post-SVM data; the buggy fallback gave 0.443, matching what was actually being reported.
>
> **Fix:** The score is now computed explicitly before the stage is recorded, the same pattern the rebalanced stage already used correctly.
>
> **Note:** The direction of every SVM finding from this period remained correct throughout; SVM still measurably increased disparity and reversed the disadvantaged group, because those findings came from raw group-rate comparisons, not this formula. Only the magnitude was wrong.

v3.6.1 → v3.6.2

> **Found by:** test_regression.py check [17], an existing check that had never actually been able to run before this validation round, because it imported from a module name that no longer existed. The first time it ran, it caught a real bug.
>
> **Root cause:** svm_validation['post_svm_bias_score'] is set once, before the gate's accept/reject decision, and the rejection branch never updated it, so after a rejected SVM result, bias_scores.final correctly fell back to the pre-SVM score, but the separate post_svm_bias_score field kept showing the discarded classifier's own (trivially low, by being useless) score.
>
> **Fix:** The gate-rejection branch now also resets svm_validation['post_svm_bias_score'] to match.
>
> **Note:** This entry directly affects how COMPAS's SVM rejection should be read in the Phase 2 report, flagged there for re-confirmation, and the v3.6.9 re-run confirmed the fix holds.

6. Version & Banner Consistency

v3.2.0 → v3.2.1

> **Found by:** A direct question from the person running the pipeline, “is it still v3.0?” after a real run, which was itself a completely reasonable thing to ask given what their own terminal was telling them.
>
> **Root cause:** Five user-facing console banners were hardcoded literal version strings, never wired to __version__, so they had printed “v3.0”/“v2.7” unchanged since the first release regardless of how many times the real version incremented.
>
> **Fix:** All five now read dynamically from __version__. Phase labels naming when a feature was introduced (e.g. “Legacy v2.7”) were deliberately left static, since making those dynamic would make them wrong in the other direction.
>
> **Verified:** A regression check now runs the pipeline and asserts the console banner's version substring matches __version__.

7. Rebalancing Methodology Evolution

v3.1.3 → v3.2.0, methodology change, not a bugfix

The resampling method changed from uniform-random add/remove of positive-outcome rows (Kamiran & Calders' "Uniform Sampling" baseline) to Preferential Sampling from the same paper: a logistic-regression ranker scores every row by closeness to the decision boundary, and rows nearest the boundary are corrected first, rather than a uniform-random pick.

Empirically confirmed on real COMPAS data: composite bias score 0.095 → 0.060 under the new method, versus 0.066 for the old uniform method.

> **Fix:** A hard minimum-group-size floor (50 samples, reusing the same threshold the pre-mitigation audit already used for its own "small group size" warnings) was added before any group becomes eligible for automatic rebalancing; previously that warning and the rebalancing engine were completely disconnected, so a group already flagged as too small to trust could still be silently resampled.
>
> **Note:** This is the same 50-sample floor referenced throughout Phase 2's report for every dataset's small-group exclusions.

v3.3.0 → v3.3.1, methodology change, not a bugfix

The default rebalance_method changed from Preferential Sampling to Reweighing. Direct comparison on the same real COMPAS input (identical starting score 0.0992): Preferential Sampling reduced it by 29.8% (to 0.0697); Reweighing reduced it by 61.7% (to 0.0380), roughly double the disparity reduction, with the same group-size floor and reversal detection applying identically to both. Preferential Sampling remains available as an explicit opt-in.

8. The Fairness Regulator, Thermostat and Fuse

v3.3.1 → v3.4.0

The most consequential single release in this history, and the origin of the framing used throughout this report and the planned BiasClean_Methodology.md.

> **Found by:** Real COMPAS data (target=is_recid, the new Reweighing default): the Age feature's disadvantaged group flipped from 'Greater than 45' (34.1% base rate, the lowest) to 'Less than 25' (59.7%, the group that started most advantaged) after rebalancing alone, before SVM ever ran.
>
> **Root cause:** The resulting gap was only about 2.5 percentage points. Reweighing's per-group targets are each independently rounded to whole rows, and near parity, that rounding noise alone is enough to flip which of several close groups is technically lowest, even though the underlying method mathematically targets exact convergence to the population mean and was never designed to overshoot past it. Overcorrection reversing the original disparity is a documented general risk in the bias-mitigation literature, not unique to this dataset or implementation.
>
> **Fix:** Two mechanisms were added, deliberately distinct in kind:

- The thermostat (proactive): _max_correction_without_crossing() computes, for any group being corrected, the maximum change that reaches the population mean without crossing past it. Both rebalancing methods now clip their own target against this ceiling before applying it, the same way a thermostat cuts the heat the instant a room hits its setpoint, rather than overshooting into overheating. Every time it engages, it is recorded in rebalance_log's "regulator_capped" list (naive target vs. what it was capped to), auditable, not silent.
- The fuse (reactive): a rebalance_gate block in audit_trail.json (accepted / reason / reversed_features), mirroring the SVM validation gate's shape. Unlike the SVM gate, this does not auto-fall back; if a reversal still gets through despite the proactive clip (e.g. one feature's rebalancing perturbing an earlier feature's row composition through row overlap, since the regulator only sees one feature at a time), it is flagged for manual review rather than silently auto-reverted. Auto-reverting one feature's specific row changes after multiple features have already been rebalanced with overlapping rows was judged a genuine row-level

rollback problem, deliberately out of scope; flag-and-review was the honest, safe boundary of what got automated.

> **Note:** Neither mechanism cares which group ends up disadvantaged; both trigger on the reversal itself, regardless of direction. This is the exact mechanism that flagged North Carolina's three-feature reversal in Phase 2, working as designed, four Phase-1 releases and two Phase-2 datasets later.

This release also added automated reversal detection as a distinct concept from magnitude tracking: every prior release only tracked whether a gap got smaller or bigger, never whether the same group stayed disadvantaged. audit_trail.json's reversal_checks block (disadvantaged group before vs. after, for both the rebalancing and SVM stages) dates to this release, and is the source for every reversal finding in the Phase 2 report.

> **Verified:** Empirically confirmed, not just unit-tested, that combining Preferential Sampling with the group-size floor prevents the specific reversal Preferential-Sampling-alone caused on COMPAS's Asian subgroup (n=32): with the floor in place, that group is excluded from rebalancing entirely, so it can no longer flip. Domain weights were also re-verified unchanged against BiasClean_v2_Methodology_Appendix.docx in this release.

9. Smart SVM, Reductions-Based Fairness Enforcement

v3.2.2 → v3.3.0

Root problem this release addresses: every SVM-related finding up to this point, worsening the composite score, reversing the disadvantaged group, collapsing to near-uniform predictions for an entire age stratum on COMPAS, and not even being reproducible run-to-run, traced back to one design flaw. The SVM stage optimized only for accuracy and had no fairness term anywhere in its own training objective.

> **Fix:** SVMFairnessEnforcer gained svm_method='fair_reduction' (the new default), wrapping the same SVC in fairlearn's ExponentiatedGradient (Agarwal, Beygelzimer, Dudík, Langford & Wallach, “A Reductions Approach to Fair Classification,” ICML 2018), an in-processing method that retrains the classifier to optimize accuracy subject to an explicit fairness constraint (equalized_odds by default, matching Hardt, Price & Srebro's NeurIPS 2016 definition; demographic_parity also available), rather than training for accuracy alone and hoping the result happens to be fair. The old 'legacy' path was kept available for direct comparison, not removed.
>
> **Note:** e.g. predict(X, random_state=...) is deterministic given a fixed seed, unlike raw SVC(probability=True), whose internal Platt-scaling cross-validation was the likely source of the run-to-run non-determinism previously observed (the same Gender reversal had flipped True/False across two identical runs).
>
> **Fix:** A mandatory validation gate was added in the orchestrator, independent of which SVM method produced the result: after the classifier stage runs, its composite score and reversal checks are computed before the result is accepted. If the classifier stage would make the composite score worse than the rebalanced baseline, or introduce a new reversal not already present after rebalancing, the result is rejected, and the pipeline falls back to the rebalanced predictions, recorded, with its exact reason, in the new svm_gate block. This is the mechanism that rejected COMPAS's SVM result in both the original baseline and the Phase 2 re-run: no single algorithm, however well-constrained, is trusted blindly.

10. Production Hardening for Sparse & Arbitrary Schemas

v3.4.0 → v3.5.0

Driven by an explicit design requirement: the pipeline needed to work correctly even with only one of the seven domain features available, and across any of BiasClean's seven domains' arbitrary column schemas, not just COMPAS's specific vocabulary.

> **Found by:** Direct instrumentation of what the real-COMPAS classifier was actually training on (rather than assuming) showed it was using ['id', 'age', 'c_days_from_compas'], a raw row identifier and a second copy of a supposedly-protected attribute, not real signal.

Root cause: A column named exactly “id” (no underscore) was never excluded from SVM features, because the substring-based id_patterns list only matches identifiers containing an underscore; a second, correct, whole-word regex already existed elsewhere in the file for the same purpose but wasn't reused here.

Fix: Both id-detection paths now use the same regex. Separately: when several raw columns are approved-mapped to the same canonical feature (real COMPAS: 'dob', 'age', and 'age_cat' all map to Age), the feature map only ever remembered one of them, silently leaving every other raw column, like the numeric 'age', unexcluded and reachable by the classifier as an ordinary predictor. UniversalBiasClean.all_protected_columns now tracks every approved-mapped raw column, not just one per feature, and this list is threaded through to feature exclusion.

Fix: A pre-flight check now skips SVM training entirely (with a clear, accurately labeled reason) if zero usable predictor columns remain after exclusions, a real possibility with a minimal dataset, instead of crashing on an empty feature matrix or training on nothing meaningful.

Fix: Explicit, deterministic degenerate-classifier detection was added to the validation gate: if fairlearn's ExponentiatedGradient collapses to predicting the same class for every row (observed directly: 7,238 of 7,238 identical predictions), the gate now rejects outright, regardless of what the score or reversal checks say. Previously this had only been caught by accident, via how the reversal check's tie-breaking happened to behave.

Verified: Regression coverage added a synthetic 1-through-7 feature-availability matrix (every possible count of the domain's protected features present, each run end-to-end), plus direct tests of each fix above.

11. Report & Regulator Verdict Consistency

v3.5.0 → v3.5.1

Found by: The first real v3.5.0 production run on COMPAS.

Root cause: audit_trail.json's rebalance_gate correctly classified Age's post-rebalancing reversal as near-parity noise (2.55-point gap, well under Age's own 7.2-point disparity threshold), exactly the distinction the v3.3.1→v3.4.0 materiality filter exists to make. But report.pdf's “Why this recommendation” section never consulted that verdict: its reversal loop flagged any detected reversal with the same alarming wording used for a genuine, actionable overcorrection, regardless of what the regulator had already concluded. The two halves of the same report contradicted each other.

Fix: The reversal loop now checks rebalance_gate's near-parity list before choosing wording, producing an accurate, calmer sentence naming the actual gap and threshold for near-parity cases. SVM-stage reversals are unaffected by this change, since the SVM gate has no materiality filter of its own; it is a hard accept/reject, so every SVM-stage reversal is still reported as genuine and actionable.

Note: This is the exact distinction the Phase 2 report relies on throughout, e.g. COMPAS's Age near-parity reversal being correctly described as non-actionable, versus North Carolina's genuine Ethnicity/Region reversals being correctly described as requiring review.

12. Statistical Leakage Detection

v3.5.1 → v3.6.0

The first of three releases found during Phase 2's own early work rather than before it began, included here because each falls before Phase 2's detailed treatment (which begins at v3.6.2) and would otherwise go undocumented.

Found by: Dataset #2 of 6 (North Carolina statewide stops, 20,286,645 real rows), auto-target-detection with all defaults, SVM enabled, the exact no-code path this whole validation exists to stress-test.

Root cause: SVM fairness enforcement reported 100.0% validation accuracy on 20.3 million rows, not plausible for a genuine classifier on a real-world fairness task at this scale. audit_trail.json confirmed pre- and post-SVM bias scores were bit-identical to 17 significant figures. This dataset has three columns that are alternate encodings of the same underlying decision as the target ('outcome', 'warning_issued', 'arrest_made'), none of which matched any existing name-based leakage pattern, all of which had been built and tuned against COMPAS's specific vocabulary during earlier hardening and didn't generalize to a differently-shaped dataset.

Fix: A statistical, name-independent leakage check was added: any remaining low-cardinality column (2–20 distinct values, present on at least half the dataset) whose per-value majority class alone predicts the target with 98% or greater purity is excluded as a near-perfect target proxy, with a printed warning naming the column and the purity that triggered it. This generalizes the same principle already used for the degenerate-classifier gate (check behavior directly and statistically, don't rely on ever-growing hand-maintained keyword lists); it is the exact mechanism that later, in Phase 2 proper, automatically excluded 'outcome' and 'raw_action_description' from North Carolina's SVM run without any dataset-specific configuration.

Note: At the time of this release, whether a genuinely uncontaminated SVM stage would still be accepted on North Carolina was flagged as an open question pending re-validation, resolved in Phase 2 (Section 3.2 of the validation report).

v3.6.0 → v3.6.1

Found by: Cross-checking report.docx (a Word conversion of North Carolina's report.pdf) against audit_trail.json, this validation round's standing rule of checking every artifact against every other rather than trusting any single file.

Root cause: audit_trail.json's deployment_decision field was correctly null (Legacy mode never runs the pre-mitigation audit that populates it). But report.pdf's Technical Detail section displayed a full, specific-looking verdict, “Recommended: No, Score: 0/100, Confidence: LOW”, for data that was never scored at all. Two independent render paths read the same underlying field; the console renderer correctly guarded against it being empty, but the HTML template that becomes report.pdf called .get(key, default) directly on an empty dictionary, and every fallback silently rendered as if it were a real result.

Fix: The HTML template now checks for an empty deployment decision the same way the console renderer already did, showing an explicit “Not computed in this run” message instead of defaulted zeros.

Note: A no-code user reading only the PDF, exactly the persona this validation tests, would reasonably have read “Score: 0/100” as a failed evaluation rather than no evaluation at all. Flagged as the same class of bug as v3.5.0→v3.5.1 (two render paths disagreeing about what the same data means), worth a broader audit of the template for the same pattern as a follow-up.

13. Cross-Cutting Patterns

The most common bug class: silent disagreement between what's stored and what's shown

Five of the fifteen releases in this range (v3.1.2, v3.1.3, v3.2.2, v3.5.1, v3.6.1) fixed some variant of the same underlying failure: a number or verdict computed correctly in one part of the pipeline failed to propagate to another part that displayed it, so the machine-readable audit trail and the human-readable report disagreed, or two fields both labeled “final” disagreed with each other. This is worth naming as a pattern in its own right for anyone maintaining this codebase going forward, not just a list of five unrelated fixes.

Real data surfaced defects synthetic testing did not

Release	What real data caught
v3.1.1	NIJ: false “no bias” result from auditing train/test split, not recidivism
v3.1.2	COMPAS: SVM made bias worse (0.464) while the report showed a 31% improvement
v3.2.2	COMPAS: the magnitude of an already-correct finding was inflated ~2.2x

v3.3.1→v3.4.0	COMPAS: rebalancing alone reversed Age's disadvantaged group
v3.4.0	COMPAS: the SVM classifier was training on a raw row ID and a duplicate protected attribute
v3.5.1→v3.6.0	North Carolina: 100% SVM validation accuracy revealed undetected target leakage

This table understates the real pattern: nearly every fix in this document was found on real data, not invented as a hypothetical edge case, consistent with the validation philosophy Phase 2 continues explicitly.

14. Bridge to Phase 2

By v3.6.1, the pipeline had: a consolidated three-file output format; hardened target and outcome-column auto-detection; internally consistent SVM-stage scoring; a two-layer overcorrection safeguard (regulator and gate) for both rebalancing and SVM; a reductions-based, gated SVM enforcement stage grounded in published fairness literature; production hardening for sparse and arbitrary schemas; and a statistical, name-independent leakage detector. This is the pipeline Phase 2's SOP describes as the v3.5.1 baseline, and it is the foundation the three further fixes found during Phase 2 itself (v3.6.7, v3.6.8, v3.6.9, documented in the Phase 2 report) were built on top of.

Recommendation: this consolidation history should be read alongside the Phase 2 validation report, not in isolation; several of Phase 2's findings (North Carolina's leakage detection, COMPAS's SVM rejection reasoning, the near-parity vs. actionable reversal distinction) are only fully legible with the Phase 1 mechanism that produces them explained, which is the gap this report closes.